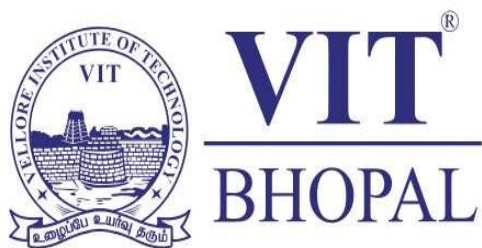


VITYARTHI – PROJECT



Project title: QUIZ BOWL – RANDOMIZED GENERAL KNOWLEDGE QUIZ

Course title: Introduction to Problem Solving & Programming

Course code: CSE 1021

Course type: Flipped course

Course credits: 4

Professor: Sivabalan KR

Slot: B21+B22+B23+E14+E21+E22

Submitted by: Nivash S

Registration number: 26MIM10032

Submission date: 29/09/2026

VITYARTHI – PROJECT

INTRODUCTION:

The **Quiz Bowl** is a Python-based command-line quiz application developed to provide a simple, interactive, and engaging way for users to test their general knowledge. The application presents multiple-choice questions to the user and allows them to select one answer from four available options: A, B, C, or D.

The project is designed as a lightweight console-based application. Instead of using a graphical interface or external database, the program stores its question bank directly inside the Python source code. The question bank contains a collection of general-knowledge questions covering areas such as geography, science, history, sports, technology, literature, mathematics, and everyday knowledge.

One of the main features of the application is **random question selection**. The program does not simply ask the questions in the order in which they are stored. It uses Python's random module and `random.sample()` to select 10 questions from the available question pool. This makes each quiz session different and prevents the user from experiencing exactly the same sequence every time.

Each question is stored using a dictionary containing three important pieces of information: the question itself, a list of four answer options, and the correct answer. These dictionaries are stored together in the `QUESTION_POOL` list.

VITYARTHI – PROJECT

The program therefore demonstrates the use of Python lists, dictionaries, functions, loops, conditional statements, string methods, exception handling, and the random module.

The main quiz functionality is implemented inside the `run_quiz()` function. The function displays a welcome message, randomly selects the questions, accepts user responses, checks whether the response is valid, compares it with the correct answer, updates the score, and finally displays the total score and percentage.

The application is intentionally simple so that it can be understood and executed easily by a beginner. It demonstrates how basic Python programming concepts can be combined to create a complete interactive application.

PROBLEM STATEMENT:

Many students and users want a simple way to practise general knowledge without requiring a complicated application or internet connection. Traditional quiz applications may contain unnecessary interfaces, registration systems, databases, advertisements, or other features that are not required for a basic learning activity.

A simple command-line quiz can solve this problem by providing questions, options, immediate feedback, and a final score in a single lightweight program.

The **Quiz Bowl** project addresses this requirement by creating an interactive Python application in which the user can answer a randomly selected set of multiple-choice questions. The

VITYARTHI – PROJECT

application automatically evaluates the answers and calculates the final result.

The project also addresses basic input-handling problems. A user may enter lowercase letters such as a instead of uppercase A, or may accidentally enter spaces before or after the answer. The program handles this by applying `.strip()` and `.upper()` to the user's input before checking the answer.

The system also validates whether the entered value belongs to the accepted choices A, B, C, or D. Invalid formats are marked as incorrect instead of causing the application to stop.

Therefore, the project provides a small but complete example of an interactive quiz system using Python programming concepts.

PROJECT OBJECTIVES:

The main objectives of the Quiz Bowl project are:

1. Develop an Interactive Quiz Application

The primary objective is to develop a console-based application that allows a user to participate in an interactive multiple-choice quiz.

2. Maintain a Structured Question Bank

The project stores questions in a structured list of dictionaries. Each question contains the question text, four options, and the correct answer.

VITYARTHI – PROJECT

3. Introduce Random Question Selection

The program uses Python's random module to select 10 questions randomly from the available question pool. This makes different quiz sessions less predictable.

4. Implement User Input Validation

The system accepts only four valid answer formats: A, B, C, and D. Other inputs are identified as invalid choices.

5. Provide Immediate Feedback

After each response, the program informs the user whether the answer is correct or incorrect. When the answer is incorrect, it displays the correct answer.

6. Calculate the Final Score

The application maintains a score counter and increases it whenever the user's answer matches the correct answer.

7. Calculate the Percentage

After all selected questions have been completed, the application calculates the percentage using the final score and the number of questions asked.

8. Demonstrate Core Python Concepts

The project demonstrates practical use of:

- Lists
- Dictionaries
- Functions
- Loops

VITYARTHI – PROJECT

- Conditional statements
- String methods
- Exception handling
- Random selection
- Formatted output

9. Create a Lightweight Offline Application

The application does not depend on external websites, databases, or APIs. The question pool and program logic are contained in the Python source file.

10. Provide a Beginner-Friendly Application

The project is designed with a simple command-line interface so that the user can understand the flow of the application without requiring advanced technical knowledge.

FUNCTIONAL REQUIREMENTS :

1. Application Startup

When the Python file is executed, the program reaches the main entry point and calls the `run_quiz()` function.

2. Welcome Display

The application displays a formatted welcome message:

WELCOME TO THE QUIZ BOWL

This indicates that the quiz has started.

VITYARTHI – PROJECT

3. Question Selection

The system determines the number of questions to ask using:

```
min(10, len(QUESTION_POOL))
```

This ensures that the program asks a maximum of 10 questions and does not attempt to select more questions than are available.

4. Randomization

The application uses:

```
random.sample(QUESTION_POOL, num_questions_to_ask)
```

This randomly selects unique questions from the question pool.

5. Multiple-Choice Display

For every selected question, the program displays the question followed by its four options.

6. User Answer Input

The user is prompted to enter an answer in the format:

A/B/C/D

7. Input Normalization

The entered answer is processed using:

```
strip().upper()
```

This removes unnecessary spaces and converts lowercase input into uppercase.

VITYARTHI – PROJECT

8. Answer Validation

The system checks whether the input belongs to:

A, B, C, D

If the answer does not match one of these choices, the program displays an invalid-choice message and marks the question as incorrect.

9. Correct Answer Verification

If the entered answer matches the answer stored in the question dictionary, the program displays:

Correct!

The score is then increased by one.

10. Incorrect Answer Handling

If the answer does not match the stored correct answer, the application displays an incorrect-answer message and identifies the correct answer.

11. Score Management

The score starts at zero before the quiz begins. It is increased only when the user provides a correct answer.

12. Question Numbering

The program uses `enumerate()` to number the questions from 1 onwards. This allows the user to understand their current position in the quiz.

VITYARTHI – PROJECT

13. Exception Handling

The program contains a try-except block around user input and answer processing. If an unexpected exception occurs, the program displays an error message and continues with the next question.

14. Quiz Completion

After all selected questions have been processed, the program displays a quiz-finished message.

15. Final Score Display

The application displays the final score in the format:

Your Final Score: score / total questions

16. Percentage Calculation

The percentage is calculated using:

Percentage = (Score / Number of Questions) × 100

The result is displayed with two decimal places.

17. Program Entry Point

The project uses:

```
if __name__ == "__main__":
```

to ensure that run_quiz() is called when the file is executed directly.

VITYARTHI – PROJECT

NON-FUNCTIONAL REQUIREMENTS:

1. Usability

The program should be easy to understand and operate. The user only needs to read the question and select one of four options.

2. Simplicity

The application avoids unnecessary complexity. It does not require a graphical user interface, account creation, or database setup.

3. Reliability

The program should continue processing questions even if an unexpected error occurs during one question.

4. Responsiveness

Since the program performs only simple in-memory operations and console input/output, there is no need for complex processing or external services.

5. Portability

The project is written in Python and can be executed in a Python-supported environment.

VITYARTHI – PROJECT

6. Maintainability

Questions are separated into a QUESTION_POOL, making it relatively easy to add or modify questions without changing the main quiz logic.

7. Offline Operation

All questions are stored locally in the program. No internet connection is required during execution.

8. Resource Efficiency

The application uses a small in-memory data structure and simple processing operations. It does not require a database or heavy third-party framework.

SYSTEM ARCHITECTURE:

The system architecture of Quiz Bowl can be divided into five logical stages.

1. Question Data Layer

The QUESTION_POOL acts as the local data source.

It contains the question text, options, and correct answer.

2. Random Selection Layer

The random.sample() function selects the questions that will be presented during the current quiz session.

VITYARTHI – PROJECT

3. User Interaction Layer

The program displays questions and receives answers using the input() function.

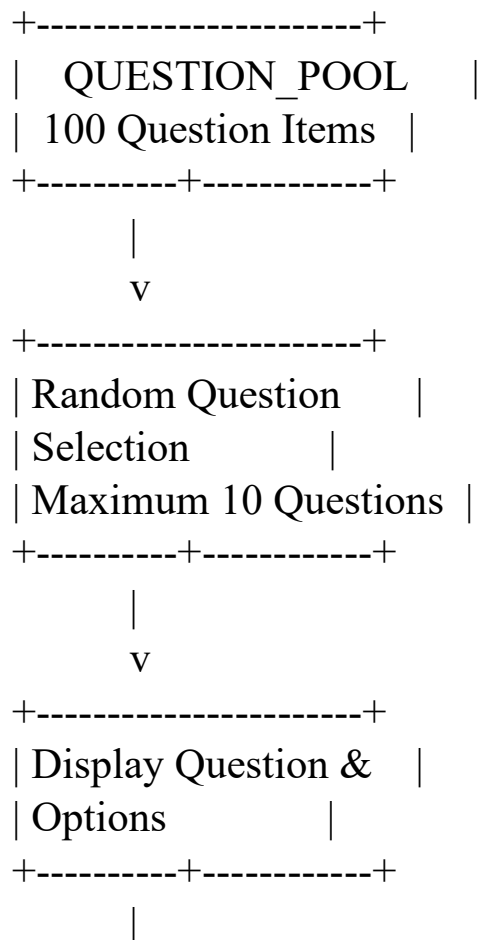
4. Validation and Evaluation Layer

The user's answer is normalized and checked against the accepted options. It is then compared with the correct answer stored in the question dictionary.

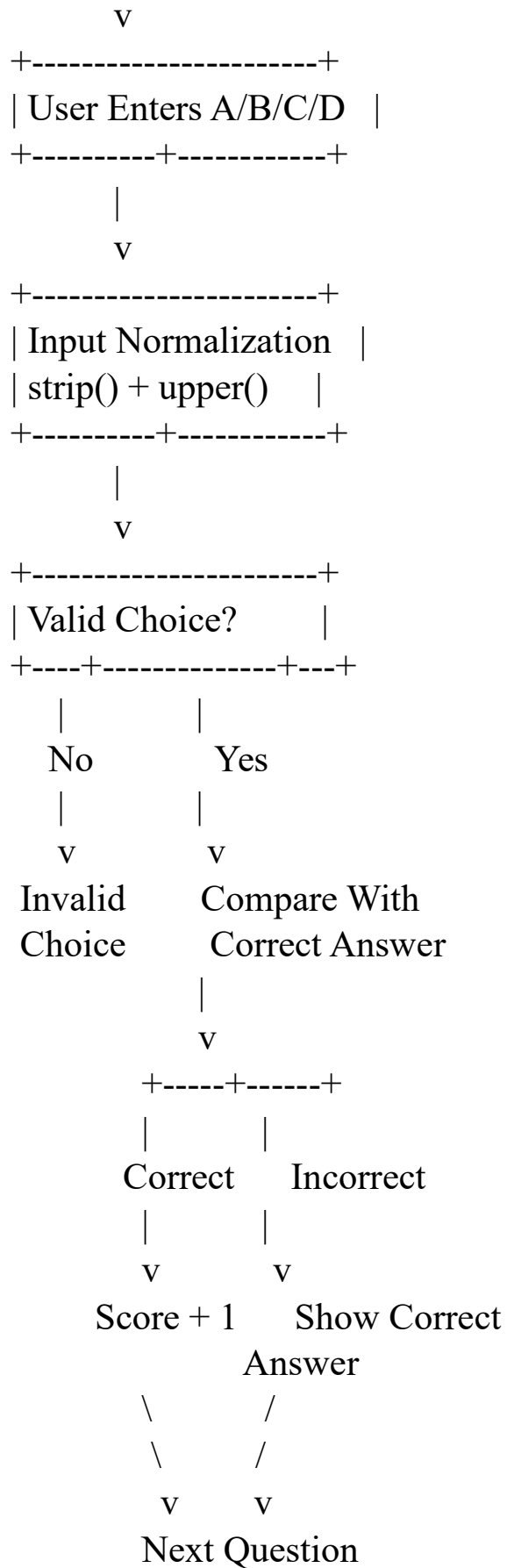
5. Result Layer

The score is displayed and converted into a percentage after all questions have been answered.

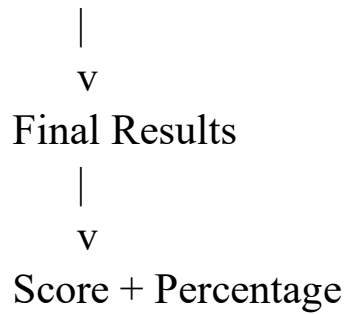
ARCHITECTURAL FLOW



VITYARTHI – PROJECT



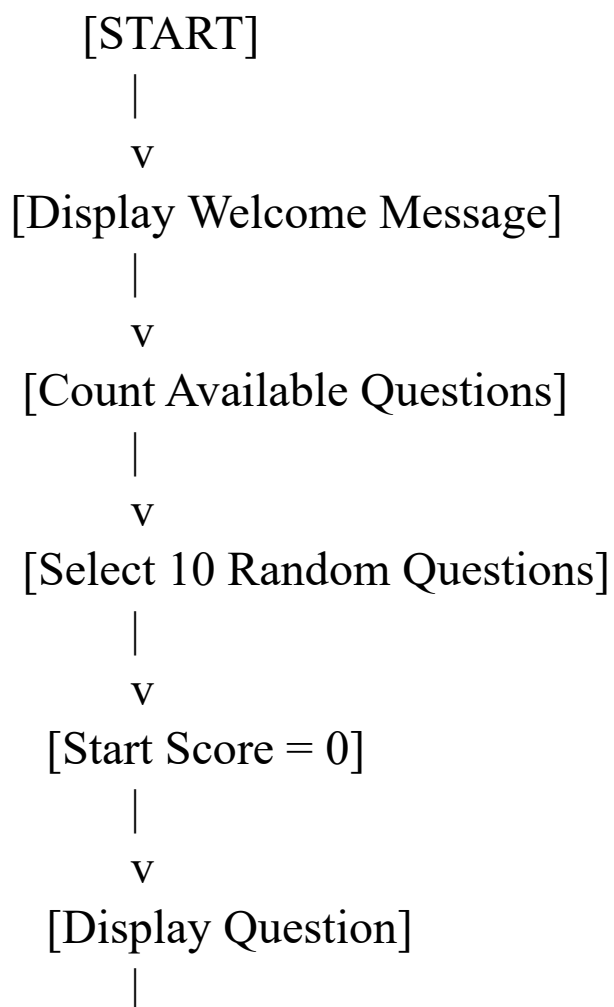
VITYARTHI – PROJECT



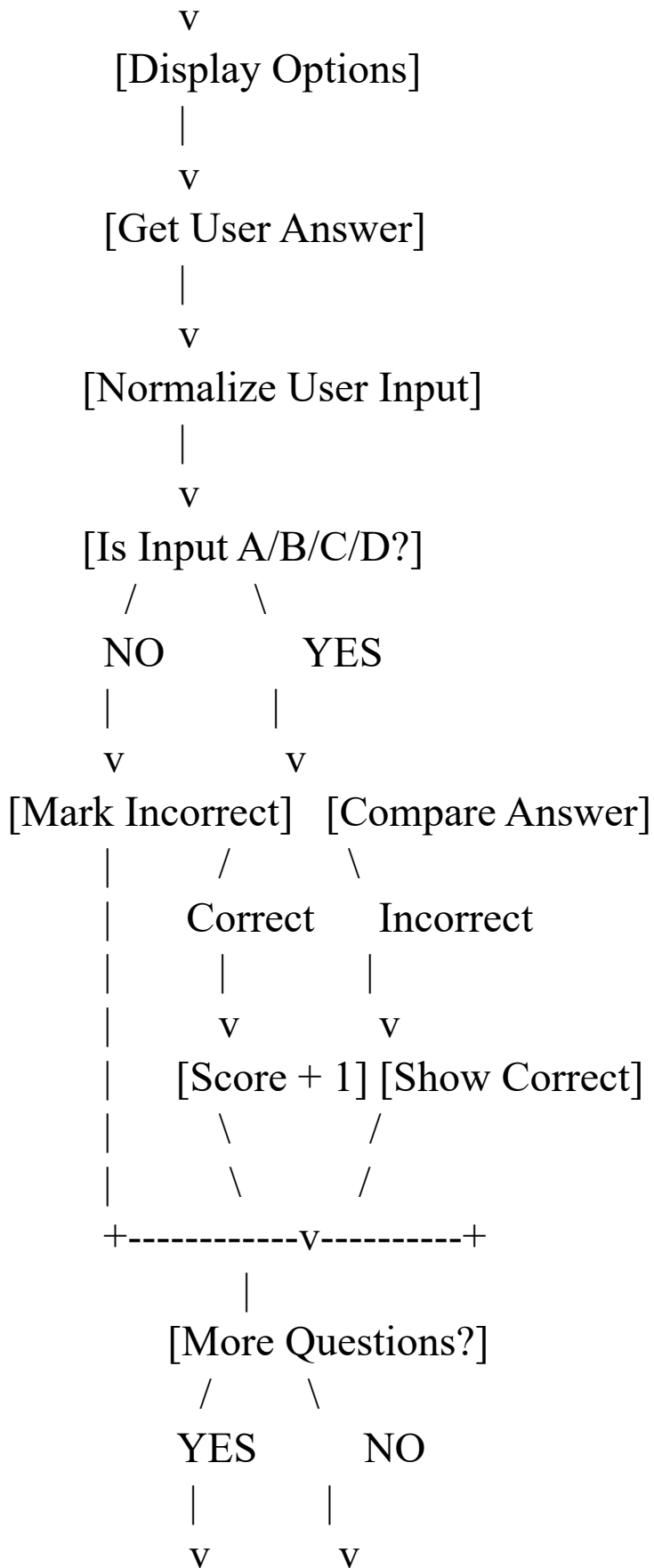
This architecture closely follows the actual execution flow implemented by `run_quiz()`.

DESIGN DIAGRAMS:

1. WORKFLOW DIAGRAM



VITYARTHI – PROJECT



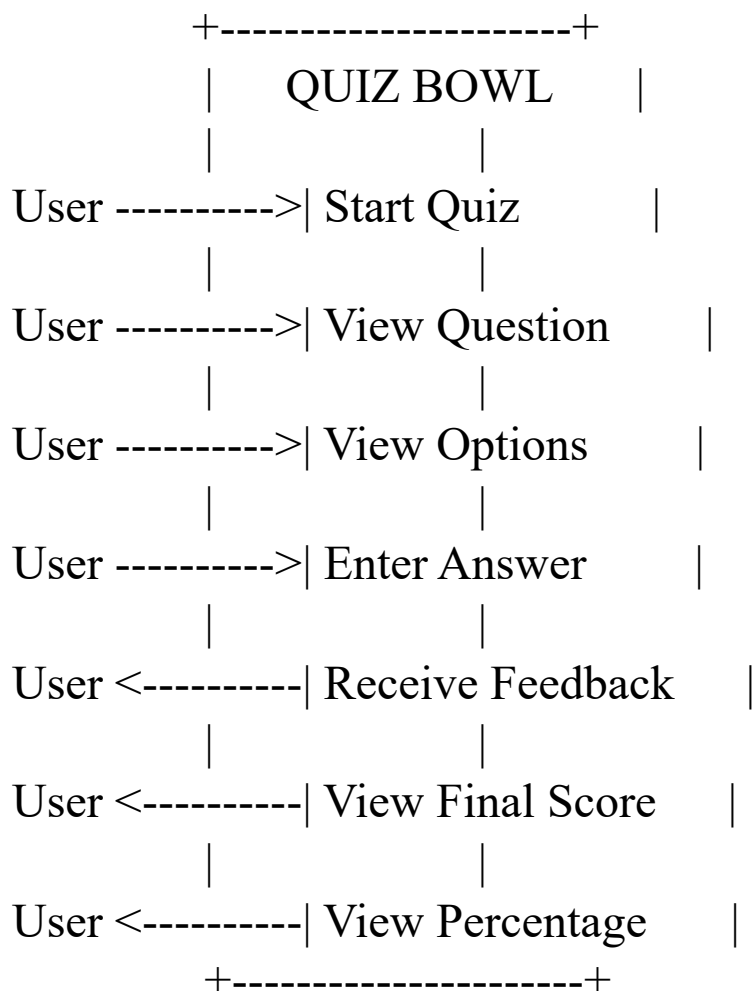
VITYARTHI – PROJECT

Next Question Final Result

|
v
[END]

USE CASE DIAGRAM

The application has one primary actor: the **User**.



VITYARTHI – PROJECT

USE CASE DESCRIPTION

Use Case 1: Start Quiz

The user runs the Python program. The application displays the Quiz Bowl welcome message and prepares the quiz.

Use Case 2: Receive Questions

The system randomly selects 10 questions from the available question pool.

Use Case 3: Answer Question

The user enters A, B, C, or D.

Use Case 4: Validate Answer

The program checks whether the input is one of the four accepted options.

Use Case 5: Evaluate Answer

The entered answer is compared with the stored correct answer.

Use Case 6: Receive Feedback

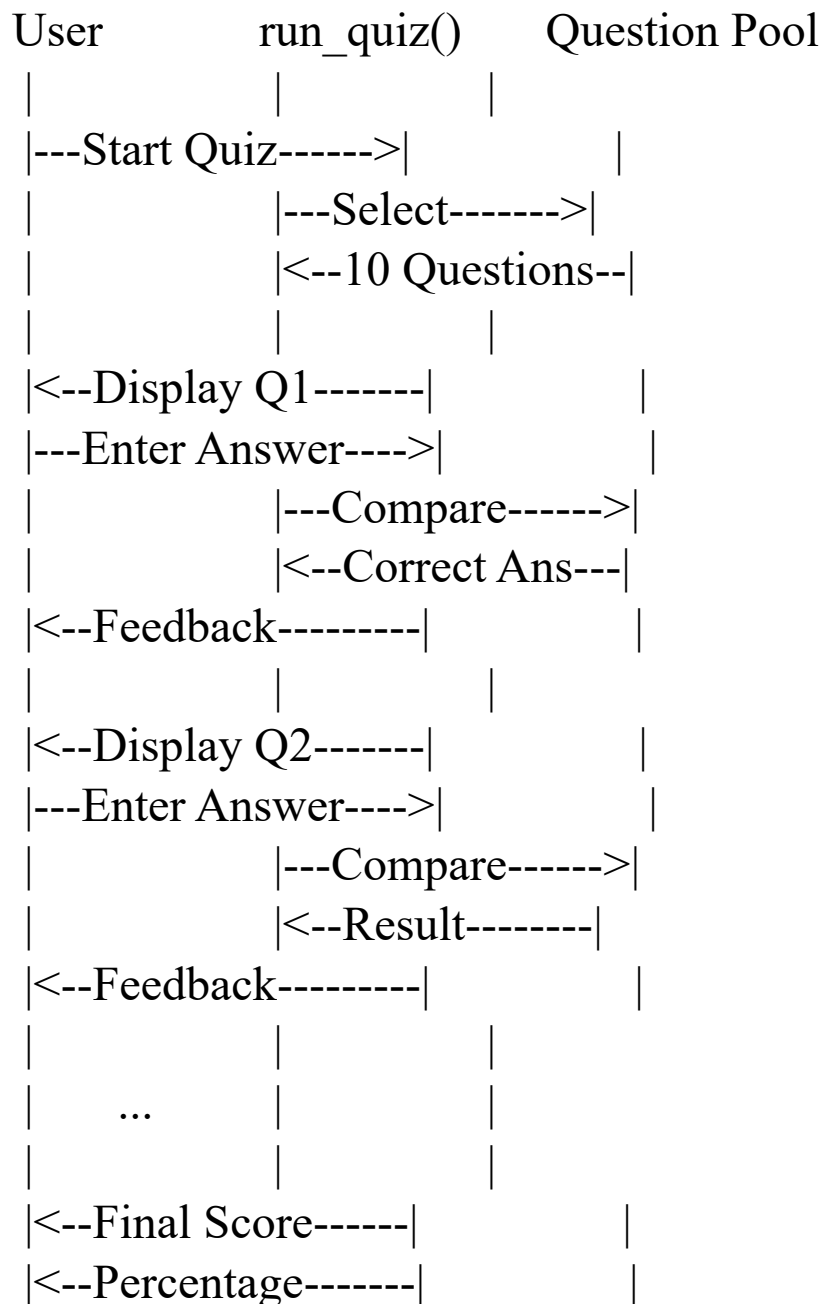
The application informs the user whether the answer was correct or incorrect.

Use Case 7: View Final Result

After the quiz is completed, the system displays the user's score and percentage.

VITYARTHI – PROJECT

SEQUENCE DIAGRAM



The sequence repeats for each of the selected questions.

VITYARTHI – PROJECT

DESIGN DECISIONS & RATIONALE:

1. Programming Language: Python

Python was used because it provides simple syntax and built-in features suitable for creating a command-line quiz application.

The project can implement lists, dictionaries, loops, functions, conditional statements, exception handling, and random selection without requiring complex code.

2. Interface Paradigm: Command-Line Interface

A command-line interface was selected because the main objective of the project is to demonstrate programming logic rather than graphical interface development.

The user can interact directly with the application through the terminal.

3. List of Dictionaries for Question Storage

The question pool uses a list containing dictionaries.

This design allows each question to keep related information together.

For every question:

Question

Options

Correct Answer

VITYARTHI – PROJECT

are stored as one logical item.

4. Random Sampling

The program uses `random.sample()` instead of simply selecting the first 10 questions.

This provides variation between quiz sessions.

It also ensures that the selected questions are unique within the session.

5. Maximum of 10 Questions

The project uses a fixed maximum of 10 questions per quiz.

This keeps the quiz short and manageable while allowing the question pool to remain much larger.

6. Input Normalization

The use of:

`strip().upper()`

was selected to make the program more user-friendly.

For example:

a

A

b

B

can be normalized into the expected uppercase format.

VITYARTHI – PROJECT

7. Basic Input Validation

The program checks:

if `user_answer` not in ['A', 'B', 'C', 'D']

This prevents unexpected answer formats from being treated as valid choices.

8. Exception Handling

A try-except block is used around the question-answer processing.

If an unexpected exception occurs, the program displays an error and proceeds to the next question rather than terminating the entire quiz.

9. In-Memory Data Storage

The project stores the question pool directly in the Python file.

This was appropriate for the current scope because the quiz does not require permanent user accounts, saved scores, or external data management.

10. Single Main Function

The major application logic is contained within `run_quiz()`.

This keeps the project simple and makes the execution flow easy to understand for a beginner.

TESTING APPROACH:

VITYARTHI – PROJECT

Testing was considered at different levels to verify that the Quiz Bowl application performs its main functions correctly.

1. Question Selection Testing

The random-selection mechanism should select a maximum of 10 questions.

The program uses:

```
min(10, len(QUESTION_POOL))
```

This ensures that the requested number does not exceed the number of available questions.

2. Question Display Testing

Each selected question should display:

- Question number
- Question text
- Option A
- Option B
- Option C
- Option D

The loop over item['options'] ensures that all four options are displayed.

3. Correct Answer Testing

A known correct option can be entered.

Expected result:

Correct!

The score should increase by one.

VITYARTHI – PROJECT

4. Incorrect Answer Testing

An incorrect but valid option can be entered.

Expected result:

Incorrect.

The correct answer should also be displayed.

5. Lowercase Input Testing

Input such as:

a

should be converted to:

A

because the application uses `.upper()`.

6. Whitespace Testing

Input such as:

B

should be processed correctly because `.strip()` removes unnecessary spaces.

7. Invalid Input Testing

Inputs such as:

X

1

YES

AB

VITYARTHI – PROJECT

are not valid quiz choices.

The application identifies them as invalid and marks the question as incorrect.

8. Score Testing

If all 10 questions are answered correctly:

Final Score: 10 / 10

Percentage: 100.00%

If five questions are answered correctly:

Final Score: 5 / 10

Percentage: 50.00%

9. Exception Testing

The try-except block ensures that unexpected exceptions during question processing are caught and reported rather than automatically terminating the quiz.

The actual source contains this defensive handling around the answer-processing section.

10. Final Result Testing

After all questions are completed, the program should display:

- Quiz Finished
- Final Score
- Percentage

CHALLENGES FACED:

VITYARTHI – PROJECT

1. Managing a Large Question Pool

A quiz requires multiple questions to provide enough variation.

Maintaining many questions manually can become difficult if the data is not organized properly. The project addresses this by storing each question as a dictionary inside a single list.

2. Random Question Selection

Simply displaying questions in their original order would make every quiz session predictable.

The use of `random.sample()` solves this problem by selecting questions randomly.

3. Maintaining the Correct Answer

Each question needs a reliable connection between its options and the correct answer.

The project stores the correct answer separately using the answer field of each dictionary.

4. Handling Different Input Formats

Users may type:

a

A

VITYARTHI – PROJECT

a

A

The program uses `.strip().upper()` to normalize these variations.

5. Invalid User Input

Users may enter something other than A, B, C, or D.

The validation condition detects these cases and prevents them from being treated as valid answers.

6. Avoiding Program Termination

An unexpected error during answer processing could potentially stop the quiz.

The project uses exception handling to catch unexpected errors and move to the next question.

7. Score Calculation

The score must be updated only when the answer is correct.

The project therefore performs the comparison first and increases the score only inside the correct-answer branch.

VITYARTHI – PROJECT

8. Percentage Calculation

The percentage needs to be calculated after all questions have been processed.

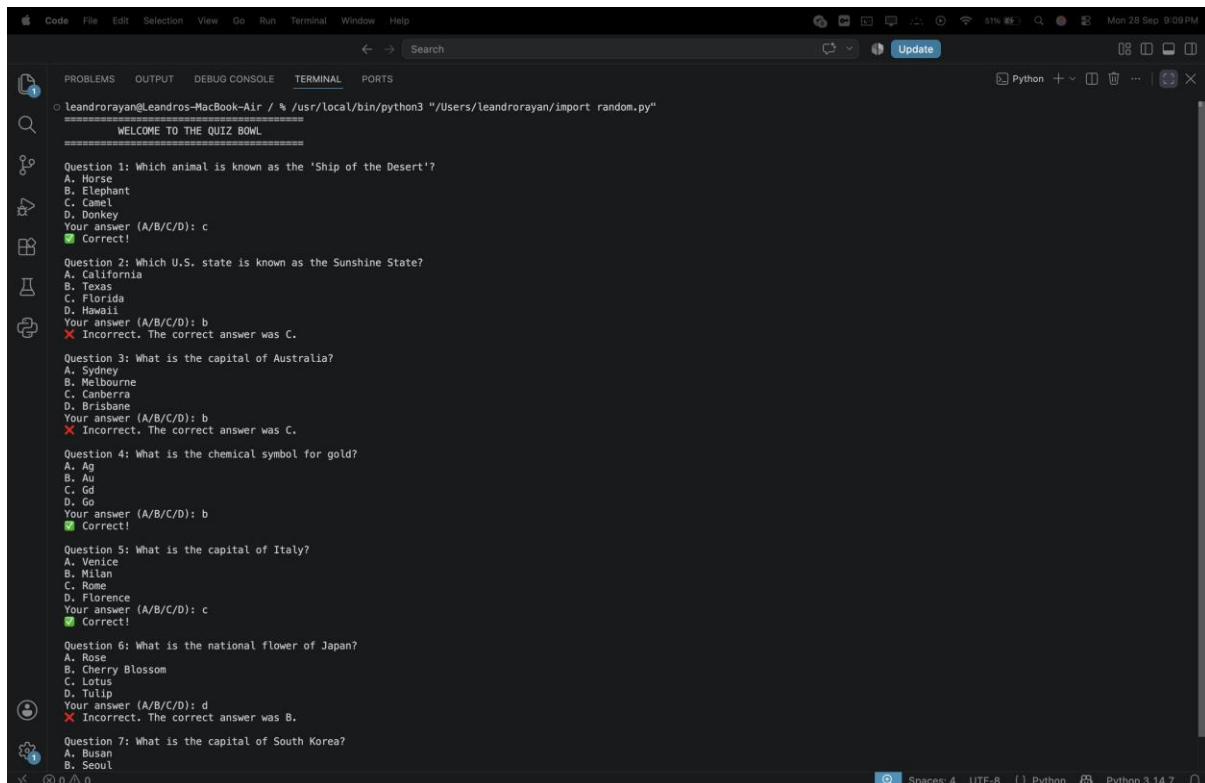
The program uses the final score and number of questions asked to calculate the result.

9. Keeping the Application Simple

Adding too many features could make the application difficult to understand.

The current design focuses on the core quiz functionality rather than introducing unnecessary external components.

OUTPUT:



```
leandrorayan@leandros-MacBook-Air / % /usr/local/bin/python3 "/Users/leandrorayan/import_random.py"
=====
WELCOME TO THE QUIZ BOWL
=====
Question 1: Which animal is known as the 'Ship of the Desert'?
A. Horse
B. Elephant
C. Camel
D. Donkey
Your answer (A/B/C/D): c
✔ Correct!

Question 2: Which U.S. state is known as the Sunshine State?
A. California
B. Texas
C. Florida
D. Hawaii
Your answer (A/B/C/D): b
✘ Incorrect. The correct answer was C.

Question 3: What is the capital of Australia?
A. Sydney
B. Melbourne
C. Canberra
D. Brisbane
Your answer (A/B/C/D): b
✘ Incorrect. The correct answer was C.

Question 4: What is the chemical symbol for gold?
A. Ag
B. Au
C. Gd
D. Go
Your answer (A/B/C/D): b
✔ Correct!

Question 5: What is the capital of Italy?
A. Venice
B. Milan
C. Rome
D. Florence
Your answer (A/B/C/D): c
✔ Correct!

Question 6: What is the national flower of Japan?
A. Rose
B. Cherry Blossom
C. Lotus
D. Tulip
Your answer (A/B/C/D): d
✘ Incorrect. The correct answer was B.

Question 7: What is the capital of South Korea?
A. Busan
B. Seoul
```

VITYARTHI – PROJECT

LEARNINGS AND KEY TAKEAWAYS:

1. Understanding Python Lists

The project demonstrates how lists can be used to store multiple related objects.

The QUESTION_POOL is a practical example of list-based data storage.

2. Understanding Dictionaries

Each question uses a dictionary to store related information.

This demonstrates how key-value structures can organize data.

3. Functions

The entire quiz process is organized inside the run_quiz() function.

This demonstrates the importance of grouping related operations into a reusable function.

4. Loops

The project uses loops to process:

- Selected questions
- Options for each question

This demonstrates how repetitive operations can be automated.

VITYARTHI – PROJECT

5. Conditional Statements

if and else conditions are used for:

- Input validation
- Correct-answer checking
- Score updates

6. Randomization

The project provides practical experience with Python's random module and random sampling.

7. String Processing

The use of:

`.strip()`
`.upper()`

demonstrates basic string manipulation and input normalization.

8. Exception Handling

The project demonstrates the use of try-except to handle unexpected errors.

9. Score Management

The project shows how a simple variable can be used as a counter and updated based on program conditions.

VITYARTHI – PROJECT

10. Percentage Calculation

The project applies a simple mathematical formula to convert the score into a percentage.

11. User Experience

Even a simple command-line application benefits from clear prompts, readable formatting, and immediate feedback.

12. Program Structure

The project demonstrates how a complete application can be created by combining data, functions, loops, conditions, and input/output operations.

KEY TAKEAWAYS

The main technical lessons from the project are:

- Structured data makes applications easier to manage.
- Randomization can improve variation in interactive programs.
- Input validation is important for reliable applications.
- Exception handling helps prevent unexpected program termination.
- Functions make program logic easier to organize.
- Immediate feedback improves user interaction.
- Simple Python features can be combined to create a complete working application.

FUTURE ENHANCEMENTS:

VITYARTHI – PROJECT

The current Quiz Bowl application provides the basic quiz functionality. Several features could be added in future versions.

1. Difficulty Levels

Questions could be separated into:

- Easy
- Medium
- Hard

The user could select the required difficulty before starting the quiz.

2. Multiple Categories

The question pool could be divided into categories such as:

- Science
- History
- Geography
- Sports
- Technology
- Mathematics
- Literature

The user could select a category before beginning the quiz.

3. Timer-Based Questions

A countdown timer could be introduced so that the user has a fixed amount of time to answer each question.

VITYARTHI – PROJECT

4. High Score System

The application could store previous scores and display the highest score achieved by the user.

5. Persistent Storage

A database or file-based storage system could be introduced to save:

- User names
- Scores
- Quiz history
- High scores
- Question statistics

6. Graphical User Interface

A GUI could be created using a Python framework such as Tkinter or another suitable GUI library.

This could replace the command-line interface with buttons, labels, and interactive screens.

7. Question Management System

A separate question-management feature could allow administrators or teachers to add, remove, or edit questions without modifying the main Python source code.

8. Larger Question Database

The current question pool can be expanded significantly to provide more variety.

VITYARTHI – PROJECT

The source itself contains a comment indicating that the base question collection can be scaled further using the same dictionary format.

9. Performance Statistics

The application could maintain detailed statistics such as:

- Correct answers
- Incorrect answers
- Accuracy by category
- Average response time
- Highest score
- Number of attempts

10. Online Question Integration

A future version could retrieve questions from an external API or online database.

This would allow the question bank to be updated dynamically instead of being permanently stored inside the source code.

11. User Profiles

Multiple users could have individual profiles and separate score histories.

12. Sound and Visual Feedback

A GUI version could provide visual or audio feedback for correct and incorrect answers.

VITYARTHI – PROJECT

REFERENCES:

- Python Software Foundation – Python Documentation.
- Python random module documentation – Random sampling and selection.
- Python documentation for built-in data structures – Lists and dictionaries.
- Python documentation for functions and control flow.
- Python documentation for exception handling using try and except.
- Python documentation for string methods such as strip() and upper().
- Project source file: **Nivash.py** – Quiz Bowl implementation.
- VITyarthi project report format/reference: **Multi-Unit Command-Line Converter** – used as the structural and content-length reference for this report.